

Domain 3

Lesson 1

Python v2

open and close

Knowing how to open a file in Python is important as it allows one to perform various operations on the file. Equally important is understanding how to close files and objects when they are no longer needed. Closing objects frees up memory space, prevents accidental altering of a file, and allows safe modification of files by other programs.

Purpose

Upon completing this project, you will become more familiar with opening and closing an object in Python.

Steps for Completion

1. Review the code, then answer the questions regarding opening and closing a file.

```
311-open.py > ...
1  message = open('311-message.txt','w')
2  message.write('Testing file for player configuration')
3  message.close()
4
5  message_test = open('311-message.txt','r')
6  print('311-message is open')
7
```

- a. On line 5, what does the 'r' indicate?

- i. read

- b. What will be the output of running this code?

- i. it opens the file writes in it then closes it then opens it reads it and print what is read

- c. What line of code would you write to close this file?

- i. message.close

- d. What are two purposes that parentheses serve in programming languages?

- i. calling functions and passing arguments and for overriding operator precedence

Project Details

Project file

N/A

Estimated completion time

5-10 minutes

Video reference

Domain 3

Topic: File Input and Output

Subtopic: open; close

Objectives covered

3 Input and Output Operations

3.1 Construct and analyze code segments that perform file input and output operations

3.1.1 open

3.1.2 close

read

When developing apps, storing information in files and reading information from files is common. For a file to be read, it must be opened in read mode and its contents saved to a variable.

Purpose

Upon completing this project, you will become more familiar with the read operation.

Steps for Completion

1. Open the **313-read.py** file from your Domain 3 Student folder.
2. On line 7, add code to read the file's contents.
3. Add a statement on the following line to print what has been read.
4. Save the file as **313-read-completed**
5. What is the r+ mode in Python?
 - a. read and write
6. When reading a file, why is it necessary to disseminate the file's contents through a print statement?
 - a. otherwise we won't be able to see what it says inside

Project Details

Project file

313-read.py

Estimated completion time

5 minutes

Video reference

Domain 3

Topic: File Input and Output

Subtopic: read

Objectives covered

3 Input and Output Operations

3.1 Construct and analyze code segments that perform file input and output operations

3.1.3 read

write

The write command in Python allows users to write data to a file and have that information ready for later use within an app or the next time an app runs. Knowing what happens with a programming language when trying to write to a file that exists or does not exist is key to managing files.

Purpose

Upon completing this project, you will better understand the write operation in Python.

Steps for Completion

1. Review the code, then answer the questions regarding writing to a file.

```
314-write.py > ...
1  message = open('314-message.txt', 'w')
2  message.write('Testing file for player
   configuration')
3  message.write('Testing file for player
   score')
4  message.close()
5
```

- a. What do lines 2 and 3 do?
 - i. writes in the files
- b. What should be written at the end of line 2 to create a new line?
 - i. \n
- c. What does the 'w' on line 1 indicate?
 - i. write mode

2. Label the following statements as true or false.

- a. false Python requires code to account for a file that does not exist.
- b. true The r+ and w+ modes of opening a file are the same.

Project Details

Project file

N/A

Estimated completion time

10 minutes

Video reference

Domain 3

Topic: File Input and Output

Subtopic: write

Objectives covered

3 Input and Output Operations

3.1 Construct and analyze code segments that perform file input and output operations

3.1.4 write

append

While the write command writes to a new file or overwrites an existing one, the append command adds text to the end of an existing file without changing the original content. The append command allows users to add new data to a file without rewriting the entire file each time.

Purpose

Upon completing this project, you will become more familiar with the append command.

Steps for Completion

1. Open the **315-append.py** file from your Domain 3 Student folder.
2. On line 3, add the code needed to indicate that a new line should be formed after the current line of text is written to the file.
3. Copy the open statement on line 1 to line 6.
4. Change the mode on line 6 from write to append.
5. On line 7, add a write statement to add the player name, **Spencer**, to the text file.
6. On line 8, add a statement to close the file.
7. Run the code and verify that the **315-message.txt** document was created.
8. Save the file as **315-append-completed**

Project Details

Project file

315-append.py

Estimated completion time

5-10 minutes

Video reference

Domain 3

Topic: File Input and Output

Subtopic: append

Objectives covered

3 Input and Output Operations

3.1 Construct and analyze code segments that perform file input and output operations

3.1.5 append

Domain 3

Lesson 2

Python v2

Check Existence

Before working on or creating a new file, it's important to check if the file already exists. Checking for a file's existence is especially useful when deciding whether to open it in write or append mode. Users can save time and effort while handling file operations by knowing whether a file exists.

Purpose

Upon completing this project, you will become more familiar with checking for the existence of a file.

Steps for Completion

1. Review the code, then answer the questions about checking for a file's existence.

```

316-check_existence.py > ...
1  import os
2
3  if not os.path.exists('316-message.txt'):
4      message = open('316-message.txt','w')
5      message.write('Testing file for player configuration\n')
6      message.write('Testing file for player score')
7      print("Configuration file made")
8      message.close()
9  else:
10     message_test = open('316-message.txt','r')
11     content = message_test.read()
12     print(content)
13     message_test.close()
    
```

- a. What does the if statement on line 3 do?
 - i. if the file no exist it makes the file and writes in it printing that it made the file
- b. If 316-message.txt does not exist, what do lines 4 through 8 do?
 - i. makes the file and writes in it also prints the thing it says it prints
- c. When would lines 10 through 13 run? What would they do?
 - i. reads and prints the file

Project Details

Project file
 N/A

Estimated completion time
 5 minutes

Video reference
Domain 3
Topic: File Input and Output
Subtopic: Check Existence

Objectives covered
3 Input and Output Operations
3.1 Construct and analyze code segments that perform file input and output operations
3.1.6 Check existence

delete

Files no longer required should be deleted so that data within them does not pose a security risk. Some potential challenges to the delete operation in Python include the file being open elsewhere, the script not having delete permissions, and corruption or disk errors.

Purpose

Upon completing this project, you will become more familiar with the delete command.

Steps for Completion

1. Review the code, then answer the questions regarding the delete operation.

```

317-delete.py > ...
1  import os
2  message_file = '317-message.txt'
3
4  if os.path.exists(message_file):
5      os.remove(message_file)
6      print("Message file removed")
7
8  else:
9      print("There was no message file to remove")
10

```

- a. What do lines 4 through 6 do?
 - i. gets rid of a file
- b. What does the else statement on lines 8 and 9 do?
 - i. if file doesn't exist then it prints that

Project Details

Project file

N/A

Estimated completion time

5 minutes

Video reference

Domain 3

Topic: File Input and Output

Subtopic: delete

Objectives covered

- 3 Input and Output Operations
 - 3.1 Construct and analyze code segments that perform file input and output operations
 - 3.1.7 delete

with Statement

Using a with statement can help reduce the amount of code needed when working with files in Python. This statement opens a file, performs an action on it, and then closes the file without telling Python to do so. By ensuring that a file is properly closed, the memory space it uses can be recovered, which can help improve device performance and reduce the risk of data corruption.

Purpose

In this project, you will practice using a with statement.

Steps for Completion

1. Open the **318-with.py** from your Domain 3 Student folder.
2. Beginning on line 6, use a with statement to rewrite the code from lines 1 through 4.
3. Comment out the code on lines 1 through 4.
4. On line 9, add a print statement that will return **file created**
5. Save the file as **318-with-completed**

Project Details

Project file

318-with.py

Estimated completion time

5 minutes

Video reference

Domain 3

Topic: File Input and Output

Subtopic: with Statement

Objectives covered

3 Input and Output Operations

3.1 Construct and analyze code segments that perform file input and output operations

3.1.8 with statement

Review 3.1

Purpose

Upon completing this project, you will better understand how file input and output work within Python.

Steps for Completion

1. Open the **318-analyze.py** file from your Domain 3 Student folder.
2. Will the code work as written now?
 - a. no
3. If the code will not work, what change needs to be made for the code to work?
 - a. change w to w+
4. Return to the video to see the answer to the question and the change that is needed to allow the code to work as intended.

Project Details

Project file

318-analyze.py

Estimated completion time

5 minutes

Video reference

Domain 3

Topic: File Input and Output

Subtopic: with Statement; Review 3.1

Objectives covered

3 Input and Output Operations

3.1 Construct and analyze code segments that perform file input and output operations

Domain 3

Lesson 3

Python v2

Read Input from Console

When developing applications, user input often needs to be captured and stored in a variable for later use. Properly capturing and storing user input ensures that an application will perform the desired actions and provide the expected output.

Purpose

Upon completing this project, you will become more familiar with reading input from console.

Steps for Completion

1. Open the **321-input.py** file from your Domain 3 Student folder.
2. Run the code.
3. Enter your first name into the console.
4. What is the response to entering your name?
 - a. says welcome and the name inputted
5. Adjust the code so there is a space after "Please enter your first name" and a comma after "Welcome to the game."
6. Run the code and enter your first name to verify your changes.
7. Save the file as **321-input-completed**

Project Details

Project file

321-input.py

Estimated completion time

5-10 minutes

Video reference

Domain 3

Topic: Console Input and Output

Subtopic: Read Input from Console

Objectives covered

3 Input and Output Operations

3.2 Construct and analyze code segments that perform console input and output operations

3.2.1 Read input from console

Print Formatted Text

Python provides various methods to control text display, which is especially important when displaying a customized message. Although some methods may be more complicated than others, they all enable users to format text to make the information display more smoothly.

Purpose

Upon completing this project, you will better understand printing formatted text.

Steps for Completion

1. Review the code, then answer the questions regarding printing formatted text.

```
322-format.py > ...
1  score = 2000
2  level = 3
3  player1 = "Tristan"
4
5  print("Player1:", player1, "Score:", score, "Level:", level)
6  print("{} has {} points and has reached level {}".format(player1, score, level))
7
```

- a. On line 6, which method is being used to format the text?

- i. .format(player1, score, level)

```
322-format.py > ...
1  score = 2000
2  level = 3
3  player1 = "Tristan"
4
5  print("Player1:", player1, "Score:", score, "Level:", level)
6  print("{}{2} has {0} points and has reached level {1}".format(score, level, player1))
7  print(f"{player1} has {score} points and has reached level {level}")
8
```

- b. On line 6, what are the numbers in curly brackets, and what will they do?

- i. they symbolize the order of how everything goes

- c. On line 7, which formatting method is being used?

- i. f-Strings

2. Which method is the preferred way of formatting strings beginning with Python 3.6?

- a. f-Strings

3. What does the Modulo operator mimic the functionality of in Python?

- a. long division

Project Details

Project file

N/A

Estimated completion time

5-10 minutes

Video reference

Domain 3

Topic: Console Input and Output

Subtopic: Print Formatted Text

Objectives covered

3 Input and Output Operations

3.2 Construct and analyze code segments that perform console input and output operations

3.2.2 Print formatted text (string.format() method, f-String method)

Use Command-Line Arguments

In some situations, a command-line argument can be used to run an app, such as when testing small blocks of code or completing files without being in a development environment. Using command-line arguments allows an app to be run multiple ways without changing the code within the app.

Purpose

Upon completing this project, you will become more familiar with command-line arguments.

Steps for Completion

1. Review the code, then answer the questions regarding command-line arguments.

```
323-command.py > ...
1 message = open('323-message.txt','w')
2 message.write('Testing file for player configuration\n')
3 message.write('Testing file for player score\n')
4 message.close()
5 print("Message file complete")
6
```

- a. What does this code do?

- i. opens and writes on 2 lines and closes and prints that it's complet

2. Open a command prompt, navigate to your Student folder, and run the code from the command line that will run the **323-command.py** file.
3. Take a screenshot of the result and save the file as **323-command-completed.png**
4. Label the following statements as true or false.
 - a. false The f-string feature is available on all versions of Python.
 - b. true One may encounter problems running code from a command line if Python is not set up as an environment variable on their system.

Project Details

Project file

323-command.py

Estimated completion time

5 minutes

Video reference

Domain 3

Topic: Console Input and Output

Subtopic: Use Command-Line Arguments

Objectives covered

3 Input and Output Operations

3.2 Construct and analyze code segments that perform console input and output operations

3.2.3 Use command-line arguments

Review 3.2

Purpose

Upon completing this project, you will have created a file from the start, encompassing pieces of what has been covered in the first three domains of this course.

Steps for Completion

1. Create a Python file that will output the following:

You can get a Rock at level 1.

You can get a Pogo Stick at level 1.

You can get a Wand at level 2.

You can get a Pogo Stick at level 2.

You can get a Wand at level 3.

You can get a Rock at level 3.

You can get a Pogo Stick at level 3.
2. Save the file as **323-items-completed** to your Domain 3 Student folder.

Project Details

Project file

N/A

Estimated completion time

15 minutes

Video reference

Domain 3

Topic: Console Input and Output

Subtopic: Use Command-Line Arguments, First Half Review

Objectives covered

3 Input and Output Operations

3.2 Construct and analyze code segments that perform console input and output operations